

RELATÓRIO 1: ANÁLISE EMPÍRICA

Comparação entre Implementações Recursivas e Iterativas

Universidade Católica de Brasília | Ciência da Computação | Análise de Algoritmos | Grupo 5 | 2026

1. Introdução

Este relatório compara as versões iterativa e recursiva dos algoritmos de Fatorial e Fibonacci. Foram analisados o tempo médio de execução $T(N)$ e a profundidade máxima da pilha $P(N)$, observando como cada abordagem se comporta quando N aumenta.

2. Metodologia

Foram desenvolvidos quatro programas em C. Cada programa recebe pela linha de comando o valor de N , a quantidade de execuções e o nome do arquivo CSV. O tempo foi medido com `clock()`, usando a média das execuções, e a pilha foi estimada pela maior distância entre endereços de variáveis locais durante as chamadas. Os resultados foram salvos nas colunas `n`, `time` e `stack`.

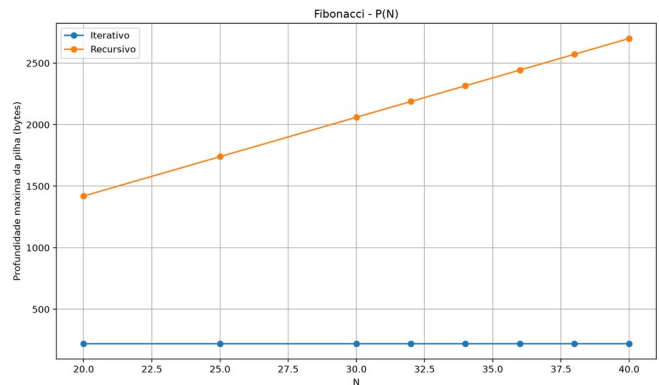
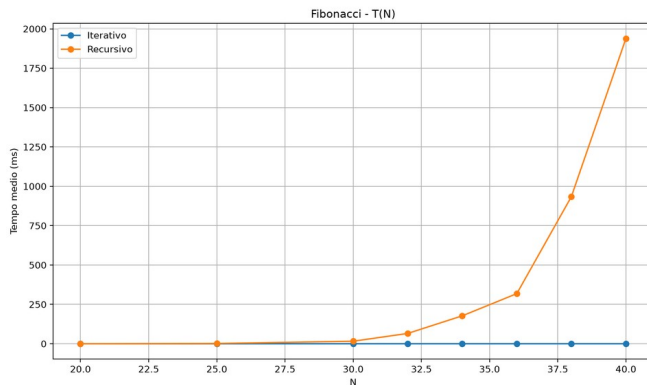
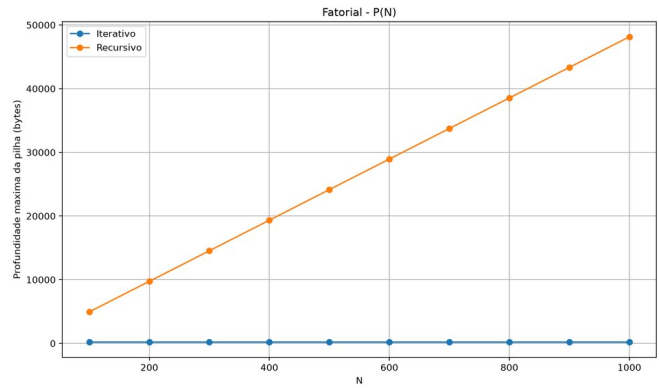
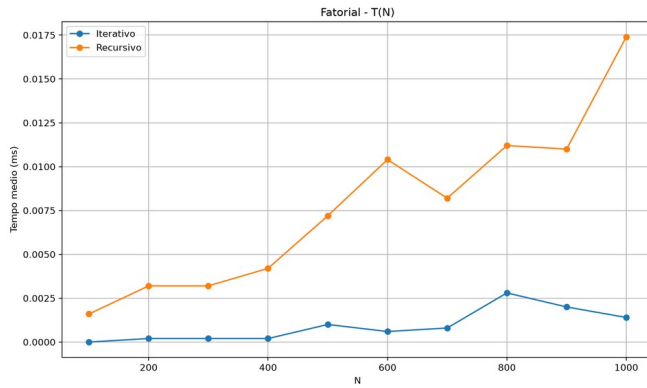
Resumo das implementações

Implementação	Lógica principal
Fatorial iterativo	Laço de 2 até N , acumulando as multiplicações.
Fatorial recursivo	Retorna $N \times \text{fatorial}(N-1)$ até atingir o caso base.
Fibonacci iterativo	Calcula os termos em sequência, guardando apenas os dois anteriores.
Fibonacci recursivo	Retorna $\text{fib}(N-1) + \text{fib}(N-2)$, gerando várias chamadas repetidas.

Testes: Fatorial com N de 100 a 1000, em passos de 100, e 5.000 execuções por valor. Fibonacci com $N = 20, 25, 30, 32, 34, 36, 38$ e 40, com 5 execuções. A faixa menor foi usada porque o Fibonacci recursivo cresce rapidamente. Os gráficos foram gerados em Python com Pandas e Matplotlib.

Observação: tempos muito pequenos podem aparecer como 0,000000 ms por limite de resolução do `clock()`. No Fatorial, N acima de 20 causa overflow em `unsigned long long`, mas isso não impede a análise de tempo e pilha.

3. Análise



Gráficos $T(N)$ e $P(N)$ de Fatorial e Fibonacci - iterativo x recursivo.

No Fatorial, as duas versões apresentaram tempos muito baixos, mas a recursiva foi, em geral, mais lenta. A diferença mais clara apareceu na pilha: a versão iterativa permaneceu em 204 bytes, enquanto a recursiva cresceu de 4.940 bytes ($N=100$) para 48.140 bytes ($N=1000$), indicando crescimento aproximadamente linear.

No Fibonacci, a diferença de tempo foi muito maior. A versão iterativa permaneceu abaixo da resolução do `clock()` na faixa testada, enquanto a recursiva passou de 15,8 ms em $N=30$ para 1.938,6 ms em $N=40$, pois repete muitos cálculos. Na pilha, a versão iterativa ficou constante em 220 bytes e a recursiva aumentou de 1.420 bytes ($N=20$) para 2.700 bytes ($N=40$).

4. Conclusão

Os resultados mostram que a abordagem iterativa foi mais eficiente nos dois problemas. No Fatorial, a principal diferença foi o crescimento da pilha na versão recursiva. No Fibonacci, além do maior uso da pilha, a recursão provocou forte aumento do tempo por repetir subproblemas. Assim, a recursão pode deixar o código próximo da definição matemática, mas seu custo de tempo e memória deve ser considerado.